TP : TMA

Module : Technologies Multimédias

Dr. EL BENANY Med Mahmoud

18 février 2026

Résumé

L'objectif de ce premier TP est de se familiariser avec les outils de simulation numérique (Python/-Numpy ou Matlab) pour générer des signaux élémentaires, comprendre l'impact de l'échantillonnage et manipuler les opérations de base (bruit, convolution).

1 TP1

1.1 Génération d'un signal sinusoïdal

On souhaite générer un signal $x(t) = A\sin(2\pi f_0 t + \phi)$.

1. Définir les paramètres : Fréquence $f_0 = 10$ Hz, Amplitude $A = 1$, Phase $\phi = 0$.
2. Choisir une fréquence d'échantillonnage $f_s = 100$ Hz et une durée de 1 seconde.
3. Créer le vecteur temps t et le signal $x(t)$.
4. Tracer le signal en fonction du temps.

1.2 Ajout de bruit et analyse

Dans la réalité, un signal est souvent corrompu par du bruit.

1. Générer un **Bruit Blanc Gaussien** (BBG) de même taille que le signal $x(t)$.
2. Créer le signal bruité $y(t) = x(t) + \text{Bruit}$.
3. Tracer sur un même graphique le signal pur et le signal bruité.

1.3 Signaux élémentaires et Convolution

1. Créer un signal porte numérique (Rect) valant 1 entre les indices $n = 20$ et $n = 40$ sur une plage de 100 points.
2. Calculer la convolution du signal porte avec lui-même.
3. Quelle est la forme du signal résultant ? Tracez-le.

2 TP2

2.1 Analyse spectrale d'un son pur

1. Générer un signal sonore composé de la somme de deux fréquences : $f_1 = 440$ Hz (La3) et $f_2 = 880$ Hz (La4) avec une fréquence d'échantillonnage $f_s = 44100$ Hz.
2. Calculer la FFT de ce signal en utilisant `numpy.fft.fft`.
3. Tracer le module de la FFT. *Attention : l'axe des abscisses doit être gradué en Hertz.*

2.2 Identification et Filtrage d'un bruit

On simule un enregistrement corrompu par un sifflement haute fréquence.

1. Ajouter au signal précédent une sinusoïde de fréquence $f_{bruit} = 5000$ Hz avec une faible amplitude.
2. Visualiser le spectre du signal bruité. Repérer le pic correspondant au bruit.
3. **Filtrage :** Créer un filtre "coupe-bande" ou "passe-bas" en mettant à zéro les coefficients de la FFT dans la zone du bruit.
4. Effectuer la Transformée de Fourier Inverse (`ifft`) pour reconstruire le signal temporel.
5. Comparer le signal original et le signal filtré (écoute et visualisation).

3 TP3

4 Partie 1 : Audio et Repliement (Aliasing)

1. Charger un fichier audio standard (wav, 44.1 kHz) avec la bibliothèque `librosa` ou `scipy.io.wavfile`.
2. Effectuer un **sous-échantillonnage** sauvage en ne gardant qu'un échantillon sur 10 (on divise f_s par 10).
3. Écouter le résultat. Que remarquez-vous sur les hautes fréquences (sons aigus, consonnes sifflantes) ?
4. **Analyse :** Comparez le spectrogramme du signal original et du signal sous-échantillonné.

5 Partie 2 : Image et Quantification

1. Charger une image en niveaux de gris (8 bits, soit 256 niveaux).
2. Réduire le nombre de niveaux de gris à 4 (2 bits) puis à 2 (1 bit).
3. **Observation :** Quel effet visuel apparaît sur les zones de dégradé (ciel, peau) ? Comment appelle-t-on ce défaut (souvent nommé *contouring* ou *banding*) ?
4. **Pixelisation :** Réduire la résolution spatiale de l'image (diviser la taille par 8) et l'afficher en l'agrandissant à sa taille originale.